
POS TECH

Tech Challenge

Tech Challenge é o projeto da fase que englobará os conhecimentos obtidos em todas as disciplinas. Esta é uma atividade que, a princípio, deve ser desenvolvida em grupo. É importante atentar-se ao prazo de entrega, pois trata-se de uma atividade obrigatória que vale 90% da nota desta fase.

Desafio

Uma oficina mecânica de médio porte, especializada em manutenção de veículos, tem enfrentado desafios para expandir seus serviços com qualidade e eficiência.

Atualmente, o processo de atendimento, diagnóstico, execução de serviços e entrega dos veículos é feito de forma desorganizada, utilizando anotações manuais e planilhas, o que gera alguns problemas, como:

- Erros na priorização dos atendimentos;
- Falhas no controle de peças e insumos;
- Dificuldade em acompanhar o status dos serviços;
- Perda de histórico de clientes e veículos;
- Ineficiência no fluxo de orçamentos e autorizações.

Diante disso, a oficina decidiu investir em um **Sistema Integrado de Atendimento e Execução de Serviços**, que permitirá aos clientes acompanhar em tempo real o andamento do serviço, autorizar reparos adicionais via aplicativo e garantir uma gestão interna eficiente e segura.

Proposta

Desenvolver a primeira versão **(MVP)** do back-end do sistema da oficina, com foco em gestão de ordens de serviço, clientes e peças, aplicando Domain-Driven Design (DDD) e garantindo boas práticas de Qualidade de Software e Segurança.

Funcionalidades obrigatórias

Fluxos principais

Criação da Ordem de Serviço (OS):

- Identificação do cliente por CPF/CNPJ;
- Cadastro de veículo (placa, marca, modelo, ano);
- Inclusão dos serviços solicitados (exemplo: troca de óleo, alinhamento);
- Possibilidade de incluir peças e insumos necessários;
- Orçamento gerado automaticamente com base nos serviços e peças;
- Envio do orçamento ao cliente para aprovação.

Acompanhamento da OS:

- Status da OS:
  - Recebida;
  - Em diagnóstico;
  - Aguardando aprovação;
  - Em execução;
  - Finalizada;
  - Entregue.
- Alteração automática dos status conforme ações no sistema;
- Permitir consulta por parte do cliente via API para acompanhar o progresso.

Gestão administrativa:

- CRUD de clientes;
- CRUD de veículos;
- CRUD de serviços;
- CRUD de peças e insumos, com controle de estoque;
- Listagem e detalhamento de ordens de serviço;

- Monitoramento do tempo médio de execução dos serviços.

Segurança e qualidade:

- Implementação de autenticação JWT para APIs administrativas;
- Validação dos dados sensíveis (CPF/CNPJ, placa de veículo);
- Testes unitários e de integração para os principais fluxos.

Requisitos técnicos

- Back-end monolítico.
- Como será um MVP; é possível criar um Monolito utilizando a arquitetura em camadas.
- A escolha do banco de dados é livre, mas é necessário justificar a preferência pelo banco utilizado.
- APIs RESTful documentadas via Swagger ou similar.
- Dockerfile para build da aplicação.
- docker-compose.yml para orquestrar ambiente completo.
- Testes automatizados com cobertura mínima de 80% nos domínios críticos.
- Configuração para execução local simples (README.md explicativo).

Entregáveis da Fase 1

A entrega deverá ser realizada por meio de **um único arquivo PDF**, contendo todas as informações e todos os links necessários para a avaliação do projeto.

A correção será realizada com base em uma análise do código-fonte, da documentação, do vídeo e das evidências apresentadas. Por esse motivo, é responsabilidade do grupo garantir que todos os links estejam corretos, acessíveis e vinculados à versão efetivamente entregue.

1. Identificação da entrega

A primeira página do PDF deverá apresentar obrigatoriamente:

- Curso: SOAT — Software Architecture;
- Fase: 1;
- Número do grupo;
- Nome completo de todos os integrantes;
- RM de todos os integrantes;
- Username utilizado no Discord;
- Linguagem e principais tecnologias utilizadas.

2. Repositório do projeto

O PDF deverá conter o link completo e clicável de todos os repositórios utilizados no projeto.

Os repositórios deverão:

- Estar públicos no GitHub durante todo o período de correção e revisão de notas;
- Possuir uma versão identificada por branch, tag ou hash de commit;
- Corresponder ao código apresentado no vídeo e na documentação;
- Conter somente arquivos necessários ao projeto, sem credenciais, senhas, tokens ou informações sensíveis;
- Possuir um `README.md` completo;
- Possuir instruções suficientes para execução local;
- Possuir os testes automatizados obedecendo ao mínimo de cobertura de 80%;
- Possuir a documentação da API por Swagger, OpenAPI ou tecnologia equivalente;
- Possuir o relatório ou resultado da análise de vulnerabilidades.

O `README.md` deverá apresentar, no mínimo:

- Objetivo do projeto;
- Descrição resumida da solução;
- Arquitetura utilizada;
- Tecnologias utilizadas;
- Pré-requisitos;
- Instruções para execução;
- Instruções para execução dos testes;
- Instruções para acesso ao Swagger ou documentação da API;
- Decisão e justificativa do banco de dados escolhido;
- Usuários, credenciais de demonstração ou procedimento de autenticação;
- Estrutura dos principais diretórios do projeto;
- Link para a documentação DDD;
- Link para o vídeo da entrega.

O grupo deverá informar no PDF a branch e o hash do commit que representam a entrega final. A avaliação deverá considerar exclusivamente essa versão.

Obs.: Lembrando que a branch main deve sempre estar protegida e tudo que chegar nela deve vir através de PR (Pull Requests).

3. Vídeo da entrega

O PDF deverá conter o link completo e clicável de um vídeo publicado no YouTube como **público**.

O vídeo deverá:

- Possuir duração máxima de 15 minutos;

- Estar disponível durante todo o período de correção e revisão de notas;
- Apresentar o projeto correspondente ao commit informado no PDF;
- Possuir áudio compreensível;
- Demonstrar o funcionamento real da solução;
- Apresentar a documentação e as decisões arquiteturais;
- Apresentar a participação ou identificação dos integrantes do grupo.

O vídeo deverá demonstrar, no mínimo:

- A proposta da solução;
- A arquitetura do projeto;
- A organização do monólito e suas camadas;
- A escolha e a justificativa do banco de dados;
- O processo de Event Storming;
- Os principais elementos de DDD utilizados;
- O fluxo de criação e acompanhamento de uma Ordem de Serviço;
- A inclusão de serviços, peças e insumos;
- A geração e aprovação do orçamento;
- A alteração dos status da Ordem de Serviço;
- A autenticação das APIs administrativas;
- A execução da aplicação;
- A documentação Swagger ou OpenAPI;
- Os testes automatizados;
- O resultado da análise de vulnerabilidades.

4. Documentação DDD

O PDF deverá conter um link direto para a documentação DDD criada no Miro ou em ferramenta equivalente.

A documentação deverá estar acessível sem solicitação manual de permissão.

Além do link externo, o grupo deverá exportar a documentação para PDF ou imagens e armazená-la no repositório, preferencialmente na estrutura:

docs/

ddd/

event-storming.pdf

diagramas.pdf

linguagem-ubiqua.md

A documentação deverá apresentar:

- Event Storming do fluxo de criação da Ordem de Serviço;
- Event Storming do acompanhamento da Ordem de Serviço;
- Event Storming da gestão de peças e insumos;
- Eventos de domínio;
- Comandos;
- Políticas ou regras de negócio;
- Atores;
- Agregados;
- Entidades;
- Objetos de valor;
- Bounded Contexts, quando identificados;

- Diagramas solicitados na disciplina de DDD;
- Linguagem Ubíqua;
- Glossário dos principais termos do domínio;
- Relação entre a documentação e a implementação realizada no código.

A documentação deverá representar a solução entregue. Diagramas genéricos, exemplos de aula ou documentação que não corresponda ao código não serão considerados como evidência completa.

5. Funcionalidades obrigatórias

O repositório deverá implementar e permitir a identificação das seguintes funcionalidades:

Criação da Ordem de Serviço

- Identificação do cliente por CPF ou CNPJ;
- Cadastro do veículo com placa, marca, modelo e ano;
- Inclusão dos serviços solicitados;
- Inclusão de peças e insumos;
- Geração automática do orçamento;
- Envio ou disponibilização do orçamento para aprovação;
- Registro da aprovação ou rejeição do orçamento.

Acompanhamento da Ordem de Serviço

A Ordem de Serviço deverá possuir os seguintes status:

- Recebida;
- Em diagnóstico;
- Aguardando aprovação;
- Em execução;

- Finalizada;
- Entregue.

O sistema deverá:

- Controlar as alterações de status;
- Aplicar as regras de transição;
- Permitir que o cliente consulte o andamento da Ordem de Serviço por meio de API.

Gestão administrativa

O sistema deverá possuir:

- CRUD de clientes;
- CRUD de veículos;
- CRUD de serviços;
- CRUD de peças e insumos;
- Controle de estoque;
- Listagem de Ordens de Serviço;
- Detalhamento de Ordens de Serviço;
- Monitoramento ou cálculo do tempo médio de execução dos serviços.

6. Requisitos técnicos obrigatórios

A solução deverá apresentar:

- Back-end monolítico;
- Organização em camadas;
- Banco de dados de livre escolha, com decisão justificada;
- APIs RESTful;
- Swagger, OpenAPI ou documentação equivalente;

- Autenticação JWT para APIs administrativas;
- Validação de CPF ou CNPJ;
- Validação da placa do veículo;
- Testes unitários;
- Testes de integração dos fluxos principais;
- Evidência de cobertura mínima de 80% nos domínios críticos;
- Instruções para execução local;
- Tratamento adequado das transações envolvidas nos principais fluxos.

Lembrem-se que estamos disponíveis no Discord para auxiliar durante todo o processo do Tech Challenge. Vamos lá?


POS TECH